

Object Oriented Programming (CoSc 2051)

Tutorial

Chapter One

Introduction to Object-Oriented Programming

Types of Programming Paradigm

Traditional development	Object-oriented system development
Focuses on the functions of the system	Centers on the object
Collection of programs (or functions)	Collection of interacting objects
Data is separate from function	Objects do things (that is, they have functionality) and they know things (they have data)
Top-down programming	Bottom-up
Structured	Modular
Not adequate	Adequate
Difficult to reuse	Reusable components
No adequate data or information hiding	Data or information hiding
high-quality programs is difficult and expensive	high-quality programs

Overview of OO Principles

- It is based on reusable components called objects.
- Objects are instances of classes and can be easily replaced, modified, and reused.
- A class is a generic representation of similar objects.
- Each object has attributes (data) and methods (functions).
- Consider an example of a real world object chair. It is a member (instance) of furniture.
- Among the many possible attributes furniture has cost, dimension, weight, location, and color. These can be applied also for table, sofa, etc. and the attributes can be reused. The same is true also for the operations.

Benefits of OO

- Better model problem domains
- Higher level of abstraction
- Seamless transition among different phases of software development
- Encouragement of good programming techniques
- Improved software development productivity
- Faster development
- Lower cost of development
- Cannot be tempered by other parts of the system
- Effective management of software complexity
- Easily maintained

Editing, Compiling and Interpreting

- Popular Java Editors
 - Notepad: you can use any simple text editor like Notepad.
 - Netbeans: A Java IDE that is open-source and free
 - Eclipse: A Java IDE developed by the eclipse open-source community
- You use the command `javac` (the Java compiler) to compile a program.
- The JVM is invoked by the `java` command to execute a Java application.
- In early Java versions, the JVM was simply an interpreter for Java bytecodes. This caused most Java programs to execute slowly, because the JVM would interpret and execute one bytecode at a time. Today's JVMs typically execute bytecodes using a combination of interpretation and so-called just-in-time (JIT) compilation.

Chapter Two

Objects and Classes

Objects and Classes

- The term object means a combination of data and function that represents some real-world entity.
- A class is a set of objects that share a common structure and a common behavior; a single object is simply an instance of a class.
- The chief role of a class is to define the properties and procedures of its instances.
- Attributes represent the state of an object.
- A method implements the behavior of an object.
- Messages are the means by which objects interact.

Cont...

- **Defining a Class:** the class keyword is followed by the name of the class.

```
class ClassName {  
    /*class body goes here*/  
}
```
- **Creating an Object:** use the new keyword

```
ClassName obj = new ClassName();
```
- **Methods and Messages:** Sending a message to an object means asking the object to execute or invoke one of its methods

```
ReturnType methodName( /* Argument list */ ) {  
    /* Method body */  
}
```

```
int x = obj.methodName();
```
- Printing to the console refers to writing the output of a Java program to the console.

Comparing and Identifying Objects

- Integer a = new Integer(1);
Integer b = new Integer(1); a == b is false
because by comparing two objects, the value of those objects is not 1. Rather it is their memory addresses
- Integer a = new Integer(1);
Integer b = a; a == b is true
- Integer a = Integer.valueOf(1);
Integer b = Integer.valueOf(1); a == b is true
- Integer a = new Integer(1);
Integer b = new Integer(1); a.equals(b) is true

Destroying Objects

- When an object is no longer needed it must be cleaned up or destroyed so that these resources are released for reuse
- Every time you want to create an object, you use the new operator to build a dynamic instance of that object.
- Java provides a feature called a garbage collector that automatically discovers when an object is no longer in use and destroys it.

Enumerated Types

- ```
public enum Spiciness {
 NOT, MILD, MEDIUM, HOT, FLAMING
}
```

This creates an enumerated type called Spiciness with five named values.
- To use an enum type
  - `Spiciness howHot = Spiciness.MEDIUM;`
- The compiler also creates an `ordinal()` method to indicate the declaration order of a particular enum constant, and a static `values()` method that produces an array of values of the enum constants in the order that they were declared

# Instance Fields

- Also called data members
- ```
class DataOnly {  
    int i;  
    double d;  
    boolean b;  
}
```
- To assign values to the fields, stating the name of the object reference, followed by a period (dot), followed by the name of the member inside the object: `objectReference.member`
- Example: `data.i = 47; data.d = 1.1; data.b = false;`
- Variables defined inside methods, constructors or blocks are called local variables.
- Instance variables are variables within a class but outside any method
- Class variables are variables declared within a class, outside any method, with the `static` keyword.

Constructors and Methods

- A constructor initializes an object when it is created. It has the same name as its class and is syntactically similar to a method. However, constructors have no explicit return type.
- Java allows two types of constructors namely:
- No argument Constructors: does not accept any parameters.
- Parameterized Constructors: accepts one or more parameters.

static, final and this

- The static keyword: When you say something is static, it means that particular field or method is not tied to any particular object instance of that class.
- final Instance Variables: You can use the keyword final to specify that a variable is not modifiable
- this keyword: Every object can access a reference to itself with keyword this (sometimes called the this reference).

Access Modifiers

- Package access: all the other classes in the current package have access to that member
- Public access: members are available to everyone.
- Private access: no one can access that member except the class that contains that member, inside methods of that class.
- Protected access: deals with a concept called inheritance. An inherited class can access a protected member as well as a public member (but not private members)

Encapsulation and Data hiding

- Encapsulation: the process of binding both attributes and methods together within a class. Through encapsulation, the internal details of a class can be hidden from outside.
- Data hiding: typically, a class is designed such that its data can be accessed only by its class methods and insulated from direct outside access. This process of insulating an object's data is called data hiding.

Chapter Three

Inheritance and Polymorphism

Inheritance

- Inheritance is the mechanism that permits new class to be created out of existing classes by extending and refining its capabilities.
- It defines an “is-a” relationship. Example from a class mammal, a number of classes can be derived such as Human, Cat, Dog, Cow, etc. Humans, Cats, Dogs, and Cows all have the distinct characteristics of mammals. In addition, each has its own particular characteristics.
- Syntax

```
public class (subclass-name) extends (existing-class-name) {  
// Changes and additions.  
}
```

Cont...

- **Types of Inheritance**
 - Single Inheritance: a subclass derives from a single super-class.
 - Multiple Inheritance: a subclass derives from more than one super-class
 - Multilevel Inheritance: a subclass derives from a super-class which in turn is derived from another class and so on.
 - Hierarchical Inheritance: a class has a number of subclasses each of which may have subsequent subclasses, continuing for a number of levels, so as to form a tree structure
- Java does not support multiple inheritance. However, a class can implement one or more interfaces, which has helped Java get rid of the impossibility of multiple inheritance.

Casting, Overriding and Overloading

- Type casting is a method or process that converts a data type into another data type in both ways Narrowing Casting (manually) and Widening Casting (automatically). The automatic conversion is done by the compiler and manual conversion performed by the programmer.
- Overriding: means to override the functionality of an existing method.
- Overloading: The ability to use the same name for two or more methods

Polymorphism

- polymorphism implies using operation in different ways, depending upon the instances they are operating upon.
- Any Java object that can pass more than one IS-A test is considered to be polymorphic.
- Polymorphism enables you to deal in generalities and let the execution-time environment handle the specifics.
- The super keyword
 - It is used to differentiate the members of superclass from the members of subclass, if they have same names.
 - It is used to invoke the superclass constructor from subclass.

Object class, Abstract class and Interface

- **The Object class:** A class that is not explicitly declared to be a of some other class is automatically made a subclass s of the standard class Object.
- **Abstract class:** You create an abstract class when you want to manipulate a set of classes through its common interface. A class containing abstract methods is called an abstract class.
- **Interface:** The interface keyword produces a completely abstract class, one that provides no implementation at all. It allows the creator to determine method names, argument lists, and return types, but no method bodies. To create an interface, use the interface keyword instead of the class keyword.

Chapter Four

Exception Handling

Exception

- An exception is a problem that arises during the execution of a program.
- When an Exception occurs the normal flow of the program is disrupted and the program terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.
- Following are some scenarios where an exception occurs.
 - A user has entered an invalid data.
 - A file that needs to be opened cannot be found.
 - A network connection has been lost in the middle of communications or the JVM has run out of memory.
- Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner.

Cont...

- Checked exceptions: an exception that is checked by the compiler at compilation-time. These exceptions cannot simply be ignored, the programmer should take care of these exceptions.
- Unchecked exceptions: an exception that occurs at the time of execution. These are ignored at the time of compilation.
- **Catching Exceptions:** A method catches an exception using a combination of the try and catch keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code.
- The code which is prone to exceptions is placed in the try block. When an exception occurs, that exception occurred is handled by catch block associated with it.
- The finally block: follows a try block or a catch block. A finally block of code always executes, irrespective of occurrence of an Exception.

Exception Methods

- `public String getMessage():` Returns a detailed message about the exception that has occurred.
- `public Throwable getCause():` Returns the cause of the exception.
- `public String toString():` Returns the name of the class concatenated with the result of `getMessage()`.
- `public void printStackTrace():` Prints the result of `toString()`.
- `public StackTraceElement [] getStackTrace():` Returns an array containing each element on the stack trace.
- `public Throwable fillInStackTrace():` Fills the stack trace of this `Throwable` object with the current stack trace.

Declaring Exceptions

- Java's built-in exceptions don't always provide the information we need. So, we sometimes need to supplement these exceptions with our own.
- A custom exception gives you more control to provide extra data about the problem.
- You can create your own exceptions in Java which are called User-defined Exceptions.
- Keep the following points in mind when writing your own exception
 - All exceptions must be a child of Throwable.
 - If you want to write a checked exception that is automatically enforced by the Handle or Declare Rule, you need to extend the Exception class.
 - If you want to write a runtime exception, you need to extend the RuntimeException class.

Cont...

- The Throws keyword: If a method does not handle a checked exception, the method must declare it using the throws keyword.
- The throws keyword appears at the end of a method's signature.
- A method can declare that it throws more than one exception, in which case the exceptions are declared in a list separated by commas.

Cont...

- Errors: These are not exceptions at all, but problems that arise beyond the control of the user or the programmer. Errors are typically ignored in your code because you can rarely do anything about an error.
- A logical error is always the symptom of a bug in application code leading to incorrect output.
- A runtime error in Java is an application error that occurs during the execution of a program.
- It is useful to catch runtime exceptions and continue program execution. To handle a runtime error, the code can be placed within a try-catch block and the error can be caught inside the catch block.

Chapter Five

Packages

Packages

- A Package can be defined as a grouping of related types providing access protection and namespace management.
- Packages are used in Java in order to prevent naming conflicts, to control access, to make searching/locating and usage easier, etc.
- Package names and directory structure are closely related.

Cont...

- Built-in Packages: These packages consist of a large number of classes which are a part of Java API.
- User-defined packages: These are the packages that are defined by the user.
- Packages are Java's way of doing large-scale design and organization. They are used both to categorize and group classes.

Cont...

- **The Import Statement:** If a class wants to use another class in the same package, the package name need not be used.
- Classes in the same package find each other without any special syntax.
- What happens if one class is not in the desired package? The class must then use one of the following techniques for referring to a class in a different package. The fully qualified name of the class can be used or the package can be imported using the import keyword and the wild card (*)

Cont...

- A class file can contain any number of import statements. The import statements must appear after the package statement and before the class declaration.
- **Static Imports:** The static import declaration is analogous to the normal import declaration.
- Where the normal import declaration imports classes from packages, allowing them to be used without package qualification, the static import declaration imports static members from classes, allowing them to be used without class qualification.

Cont...

- The import provides accessibility to classes and interface whereas static import provides accessibility to static members of the class.
- **CLASSPATH:** The full path to the classes directory, is called the class path, and is set with the CLASSPATH system variable.
- Both the compiler and the JVM construct the path to your .class files by adding the package name to the class path. A class path set via the operating system, and sometimes by the installation program that installs Java.

Cont...

- **Defining Packages:** Programmers can define their own packages to bundle group of classes/interfaces, etc.
- It is a good practice to group related classes implemented by you so that a programmer can easily determine that the classes, interfaces, enumerations, and annotations are related.
- Since the package creates a new namespace there won't be any name conflicts with names in other packages. Using packages, it is easier to provide access control and it is also easier to locate the related classes.

Cont...

- While creating a package, you should choose a name for the package and include a package statement along with that name at the top of every source file that contains the classes, interfaces, enumerations, and annotation types that you want to include in the package.
- The package statement should be the first line in the source file (except for comments and white space, of course).
- There can be only one package statement in each source file, and it applies to all types in the file. If a package statement is not used then the class, interfaces, enumerations, and annotation types will be placed in the current default package.

Cont...

- It is a good practice to use names of packages with lower case letters to avoid any conflicts with the names of classes and interfaces.
- **Package Scope:** The default access has no keyword, but it is commonly referred to as package access. It means that all the other classes in the current package have access to that member, but to all the classes outside of this package, the member appears to be private.
- Package access allows you to group related classes together in a package so that they can easily interact with each other. When you put classes together in a package, thus granting mutual access to their package-access members, you “own” the code in that package.

Chapter Six

Data Structures

The set

- It used to create the mathematical set.
- An unordered collection or list in which duplicates are not allowed is referred to as a collection interface. The set interface use collection interface's methods to avoid the insertion of the same elements.
- Sets contain no pair of elements $e1$ and $e2$ such that $e1.equals(e2)$, and at most one null element.

Set Implementation

- The Java platform contains three general-purpose Set implementations: HashSet, TreeSet, and LinkedHashSet.
- HashSet, which stores its elements in a hash table, is the best-performing implementation;
- On the Set, we can perform all the basic mathematical operations like intersection, union and difference.

Cont...

- In set, `addAll()` method is used to perform the union, `retainAll()` method is used to perform the intersection and `removeAll()` method is used to perform difference.
- There are several methods available in the set interface which we can use to perform a certain operation on our sets.
- The `add()` method insert a new value to the set. The method returns true and false depending on the presence of the insertion element. It returns false if the element is already present in the set and returns true if it is not present in the set.

Cont...

- The `addAll()` method appends all the elements of the specified collection to the set.
- The `clear()` method removes all the elements from the set. It doesn't delete the reference of the set. It only deletes the elements of the set.
- The `contains()` method is used to know the presence of an element in the set. Its return value is true or false depending on the presence of the element.

Cont...

- The `containsAll()` method is used to check whether all the elements of the collection are available in the existing set or not. It returns `true` if all the elements of the collection are present in the set and returns `false` even if one of the elements is missing in the existing set.
- The `isEmpty()` method is used to identify the emptiness of the set . It returns `true` if the set is empty and returns `false` if the set is not empty.
- The `iterator()` method is used to find the iterator of the set. The iterator is used to get the element one by one.

Cont...

- The `remove()` method is used to remove a specified element from the Set. Its return value depends on the availability of the element. It returns `true` if the element is available in the set and returns `false` if it is unavailable in the set.
- The `removeAll()` method removes all the elements of the existing set from the specified collection.
- The `retainAll()` method retains all the elements from the set specified in the given collection.
- The `size()` method returns the size of the set and the `Object[] toArray()` method is used to create an array with the same elements of the set.

The List

- List in Java provides the facility to maintain the ordered collection. It contains the index-based methods to insert, update, delete and search the elements. It can have the duplicate elements also. We can also store the null elements in the list.
- The List interface is found in the `java.util` package and inherits the Collection interface.
- It is a factory of ListIterator interface. Through the ListIterator, we can iterate the list in forward and backward directions

List Implementation

- The implementation classes of the List interface are ArrayList, LinkedList, Stack, and Vector. The ArrayList and LinkedList classes are widely used in Java programming.
- The add(i,e) or add(e) method is used to insert the specified element at the specified position in a list or at the end.
- The get() method returns the element at the given index, whereas the set() method changes or replaces the element.

Cont...

- The `Collections.sort()` method used to sort the list element.
- `ListIterator` Interface is used to traverse the element in a backward and forward direction.
- The `remove()` method is used to remove the element present at the specified position in the list.
- The `removeAll()` or `clear()` methods are used to remove all the elements from the list.

The stack

- The stack is a linear data structure that is used to store the collection of objects. It is based on Last-In-First-Out (LIFO).
- The stack data structure has the two most important operations that are push and pop. The push operation inserts an element into the stack and pop operation removes an element from the top of the stack.
- Empty Stack: If the stack has no element is known as an empty stack. When the stack is empty the value of the top variable is -1. When we push an element into the stack the top is increased by 1. When we pop an element from the stack the value of top is decreased by 1. When the stack is full the value of the top variable is N-1.

Stack Implementation

- In Java, Stack is a class that falls under the Collection framework that extends the Vector class.
- The empty() method of the Stack class check the stack is empty or not. If the stack is empty, it returns true, else returns false.
- The push() method inserts an item onto the top of the stack. It passes a parameter item to be pushed into the stack.

Cont...

- The pop() method removes an object at the top of the stack and returns the same object. It throws EmptyStackException if the stack is empty.
- The peek() method looks at the element that is at the top in the stack. It also throws EmptyStackException if the stack is empty.
- The search() method searches the object in the stack from the top. It returns the object location from the top of the stack. If it returns -1, it means that the object is not on the stack.

Cont...

- The `size()` method used to find the size of the stack. It returns the total number of elements (size of the stack) in the stack.
- The `iterator()` method used to fetch the elements of the stack.

The Queue

- Java Queue interface orders the element in FIFO (First In First Out) manner. In FIFO, first element is removed first and last element is removed at last.
- The PriorityQueue class provides the facility of using queue. But it does not orders the elements in FIFO manner.
- It inherits AbstractQueue class.

Queue Implementation

- Some of the frequently used Queue implementation classes are LinkedList, PriorityQueue, ArrayBlockingQueue, DelayQueue, LinkedBlockingQueue, PriorityBlockingQueue etc.
- AbstractQueue provides a skeletal implementation of the Queue interface to reduce the effort in implementing Queue

Cont...

- The `add()` method is used to insert the specified element into this queue and return `true` upon success.
- The `offer()` method is used to insert the specified element into this queue.
- The `remove()` method is used to retrieve and removes the head of this queue.

Cont...

- The `poll()` method is used to retrieve and remove the head of this queue, or returns null if this queue is empty.
- The `element()` method is used to retrieve, but does not remove, the head of this queue.
- The `peek()` method is used to retrieve, but does not remove, the head of this queue, or returns null if this queue is empty

Map/Dictionary

- A map contains values on the basis of key, i.e. key and value pair. Each key and value pair is known as an entry.
- A Map contains unique keys. A Map is useful if you have to search, update or delete elements on the basis of a key.
- are two interfaces for implementing Map in java: Map and SortedMap, and three classes: HashMap, LinkedHashMap, and TreeMap.

Cont...

- A Map doesn't allow duplicate keys, but you can have duplicate values.
- HashMap and LinkedHashMap allow null keys and values, but TreeMap doesn't allow any null key or value.
- A Map can't be traversed, so you need to convert it into Set using `keySet()` or `entrySet()` method.

Cont...

- The `put()` method is used to insert an entry in the map.
- The `remove()` method is used to delete an entry for the specified key.
- The `keySet()` or `entrySet()` methods are used to return the Set view containing all the keys.
- The `clear()` method is used to reset the map

Cont...

- The equals() method is used to compare the specified Object with the Map.
- The isEmpty() method returns true if the map is empty; returns false if it contains at least one key.
- The size() method returns the number of entries in the map.